

Управление транзакциями в приложениях

Мельников В.М., 2017

Содержание

- Механизмы управления транзакциями в JDBC
- Механизмы управления транзакциями в J2EE
 - Стандарт EJB 3.1
 - Декларация транзакций
 - Обработка ошибок
 - Стандарт JTA 1.1 (X/Open XA)
 - Управление распределенными транзакциями
- Управление транзакциями с использованием AOP (Aspect-Oriented Programming)
 - Пример использования SpringFramework

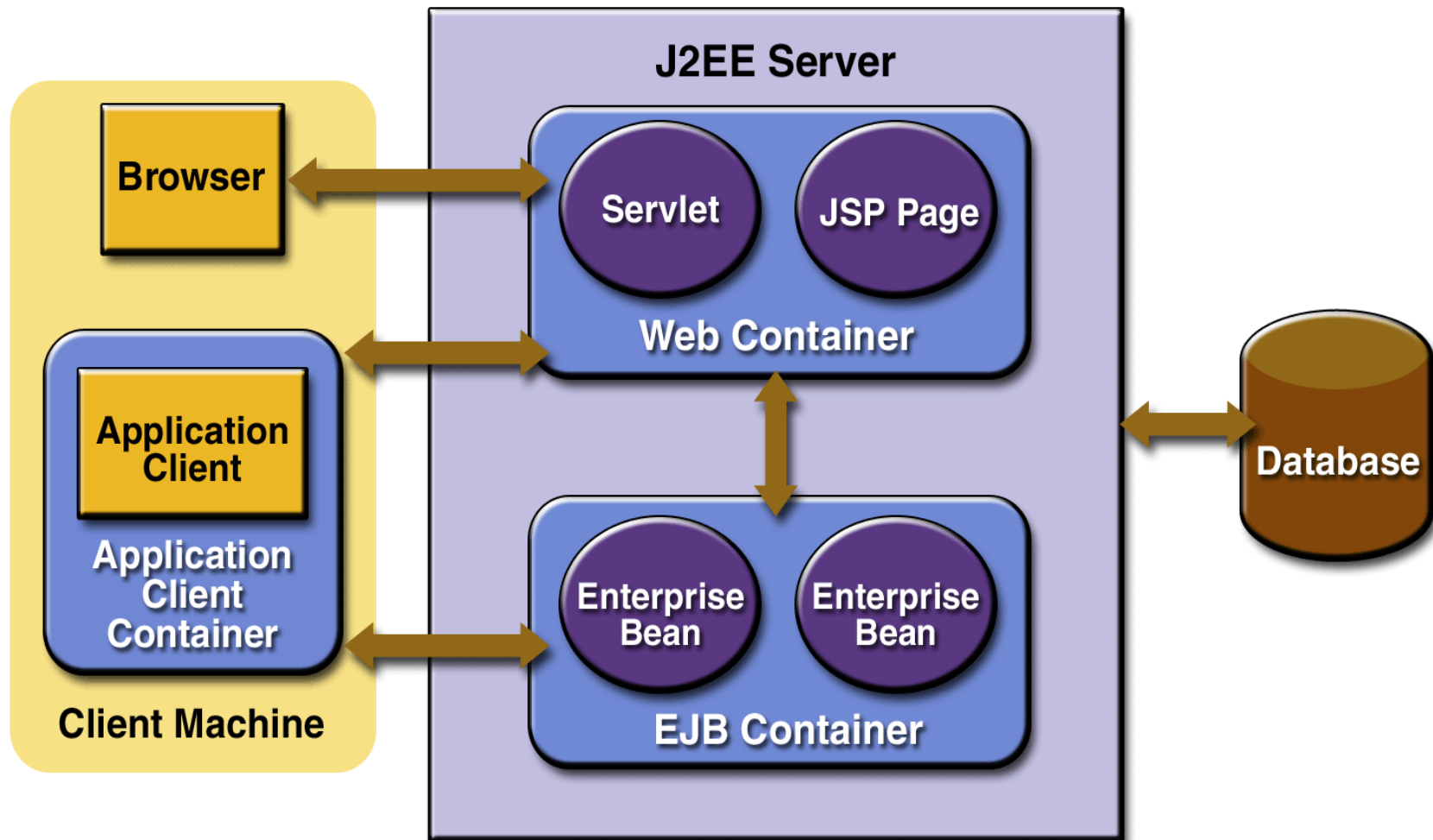
JDBC. Управление транзакциями

```
try {  
    connection.setAutoCommit(false);  
    .....  
    connection.commit();  
}  
catch(Exception e) {  
    connection.rollback();  
}  
finally {  
    connection.close();  
}
```

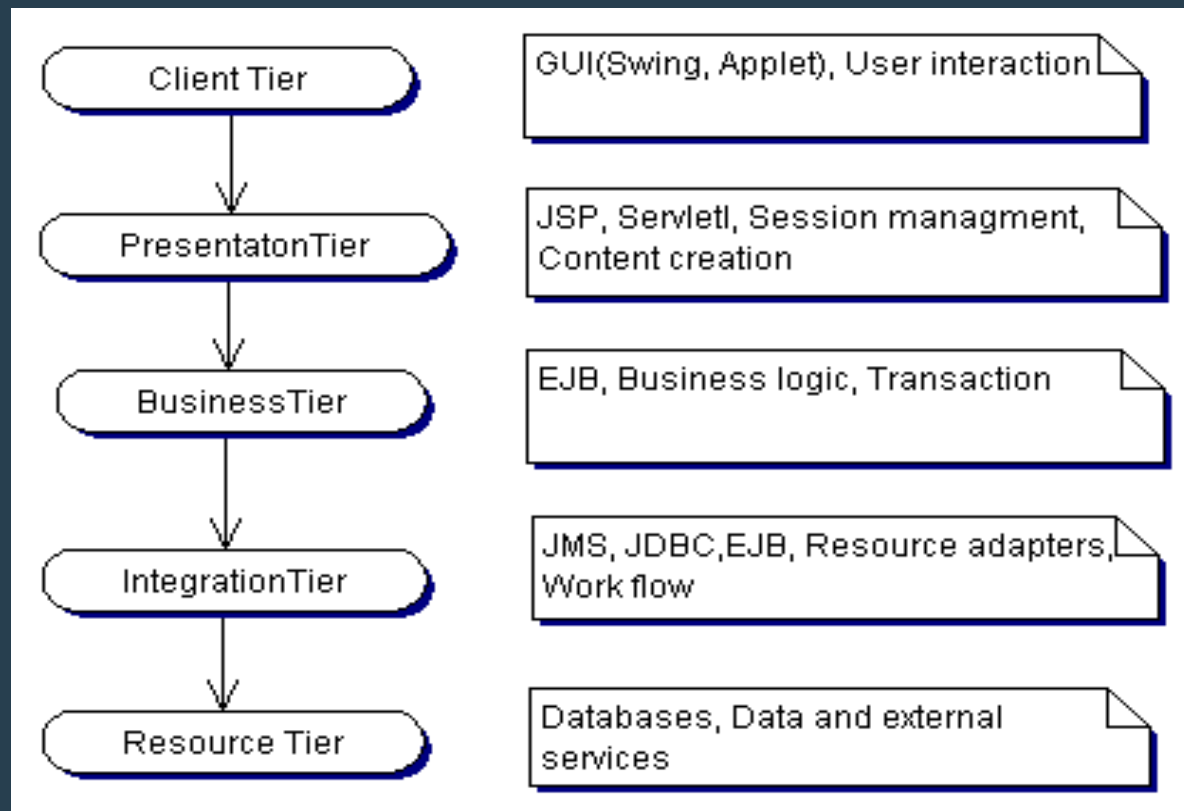
Сервер приложений

- ❑ Программная платформа для функционирования процедур(программ)
- ❑ Представляется для разработчика как большой набор готовых стандартных компонентов
- ❑ Является контейнером для бизнес-логики приложений
- ❑ Как правило являются одним из звеньев 3-х уровневой архитектуры

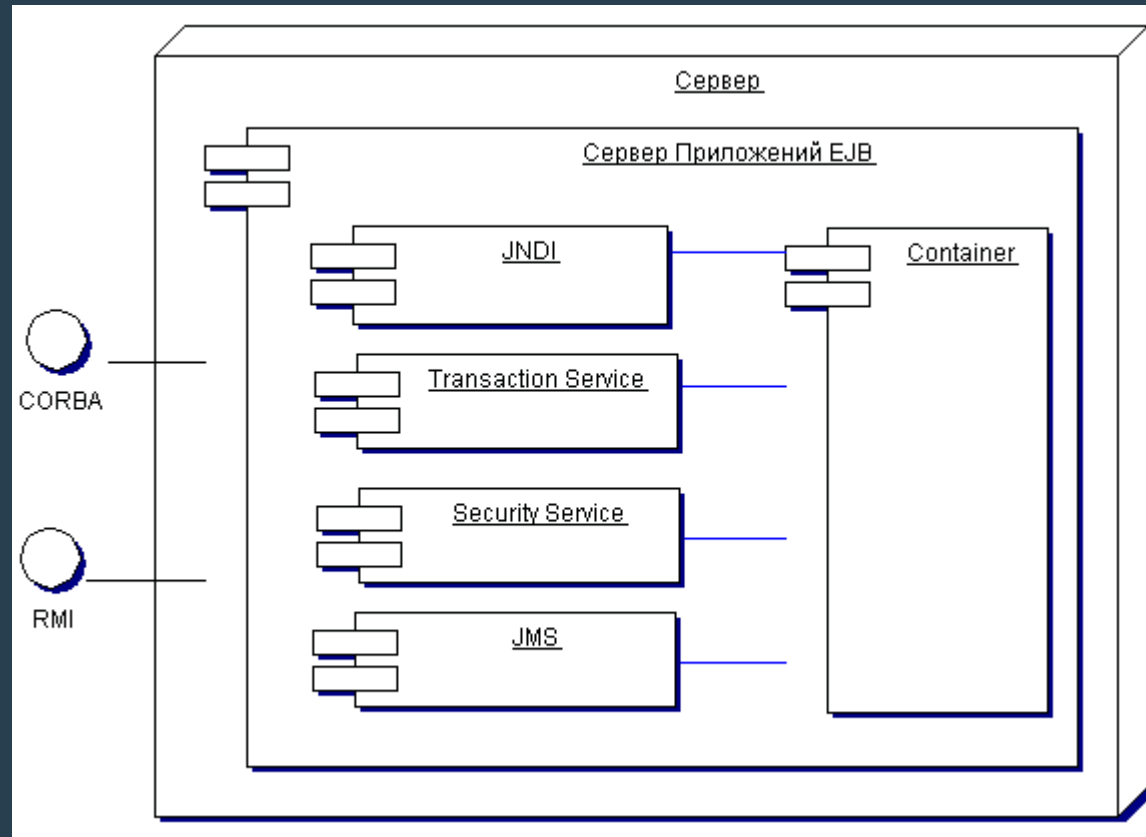
3-х уровневая архитектура



5-и уровневая архитектура в Java



ЕJB контейнер



Enterprise Java Bean

Типы ЕJB

Entity Bean (Java Persistence API)

- Отображение данных из реляционной БД на объекты Java

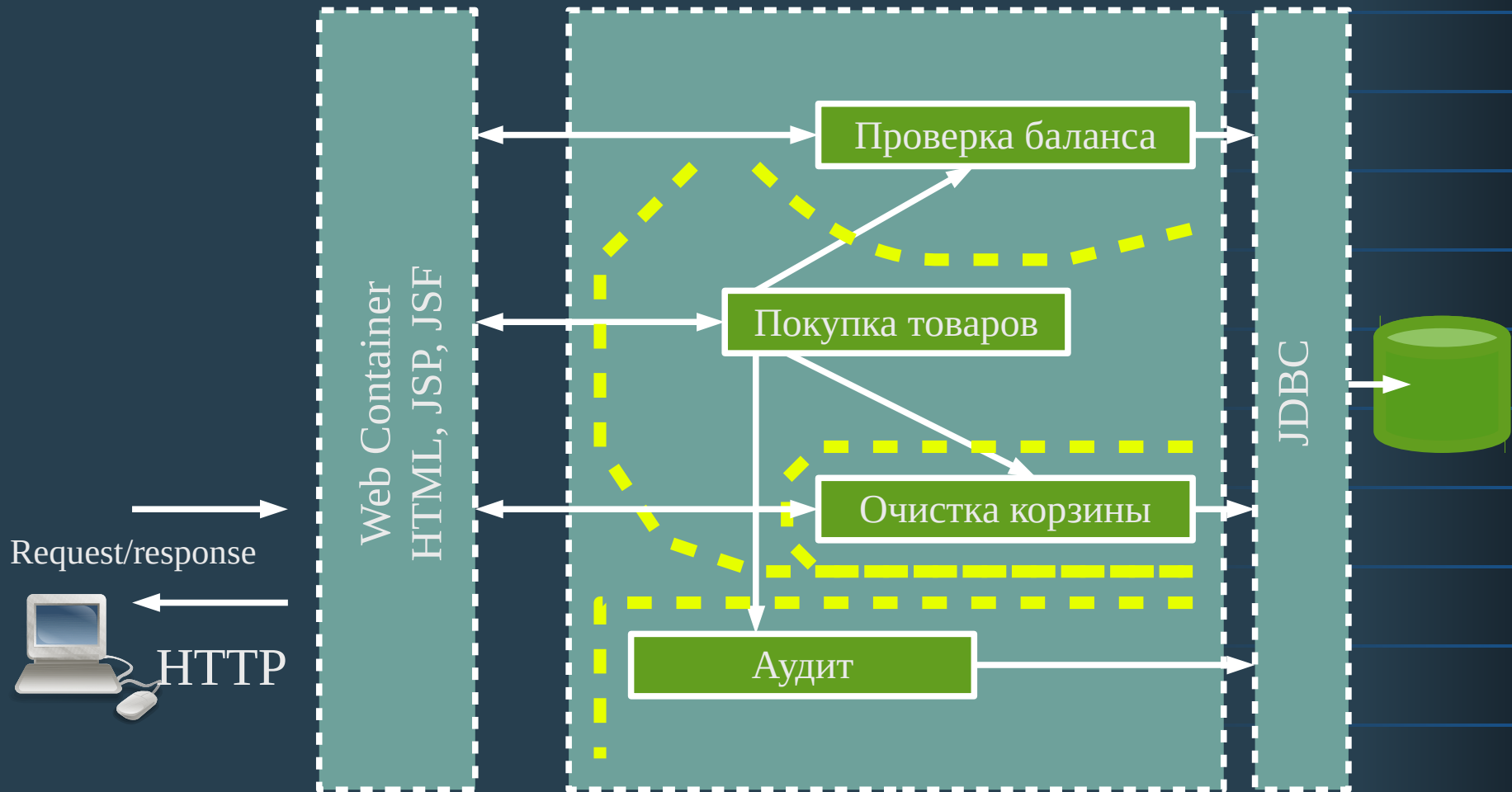
Session Bean (Stateless, Statefull)

- Содержат бизнес-логику обработки данных
- Без хранения состояния, с сохранением состояния
- Может переиспользоваться между вызовами клиентов или быть привязанным к одному из клиентов
- Состояние может сохраняться на диске

Message Driven Bean

- Содержит бизнес-логику обработки при поступлении сообщения в именованную очередь
- Используется для асинхронной обработки и интеграции

Декомпозиция приложения



ЕJB. Управление транзакциями



@Stateless

@TransactionManagement(TransactionManagementType.CONTAINER)

class ShopService {

.....



Один вызов метода – одна транзакция



@Statefull

@TransactionManagement(TransactionManagementType.BEAN)

class ComplexEventProcessor {

 @Resource

 private UserTransaction userTransaction;

.....



Началом и завершением транзакции управляет класс

ЕJB. Границы транзакций

❶ Управление аннотациями

- ❶ @TransactionAttribute(REQUIRED)

```
void buyItems(...) {  
    .....  
}
```

❷ Типы атрибутов

- ❶ SUPPORTS

- ❶ REQUIRED

- ❶ REQUIRED_NEW

- ❶ NOT_SUPPORTED

- ❶ MANDATORY

- ❶ NEVER

ЕJB. Границы транзакций

Атрибут	Транзакция запущена клиентом	Транзакция не запущена клиентом
SUPPORTS	Используем транзакцию клиента	Не используем транзакции
REQUIRED	Используем транзакцию клиента	Запускаем новую транзакцию
REQUIRED_NEW	Запускаем новую транзакцию	Запускаем новую транзакцию
MANDATORY	Используем транзакцию клиента	Генерируем исключение
NEVER	Генерируем исключение	Не используем транзакции
NOT_SUPPORTED	Приостанавливаем транзакцию клиента	Не используем транзакции

ЕJB. Обработка исключений

- ❶ System Exceptions – автоматический откат транзакции
 - ❶ RuntimeException, RemoteException
 - ❶ Оборачиваются в EJBException
- ❶ Application Exception
 - ❶ Checked Exception – декларируется как часть ожидаемого поведения
 - ❶ По умолчанию не приводят к откату транзакции
 - ❶ Для вызывающей стороны приходят в неизменном виде
 - ❶ Допускается выполнение отката с использованием аннотации у исключения
 - ❶ @ApplicationException(rollback=true)
Class MyAppException extends Exception {
.....

Java Transaction API (JTA 1.1)

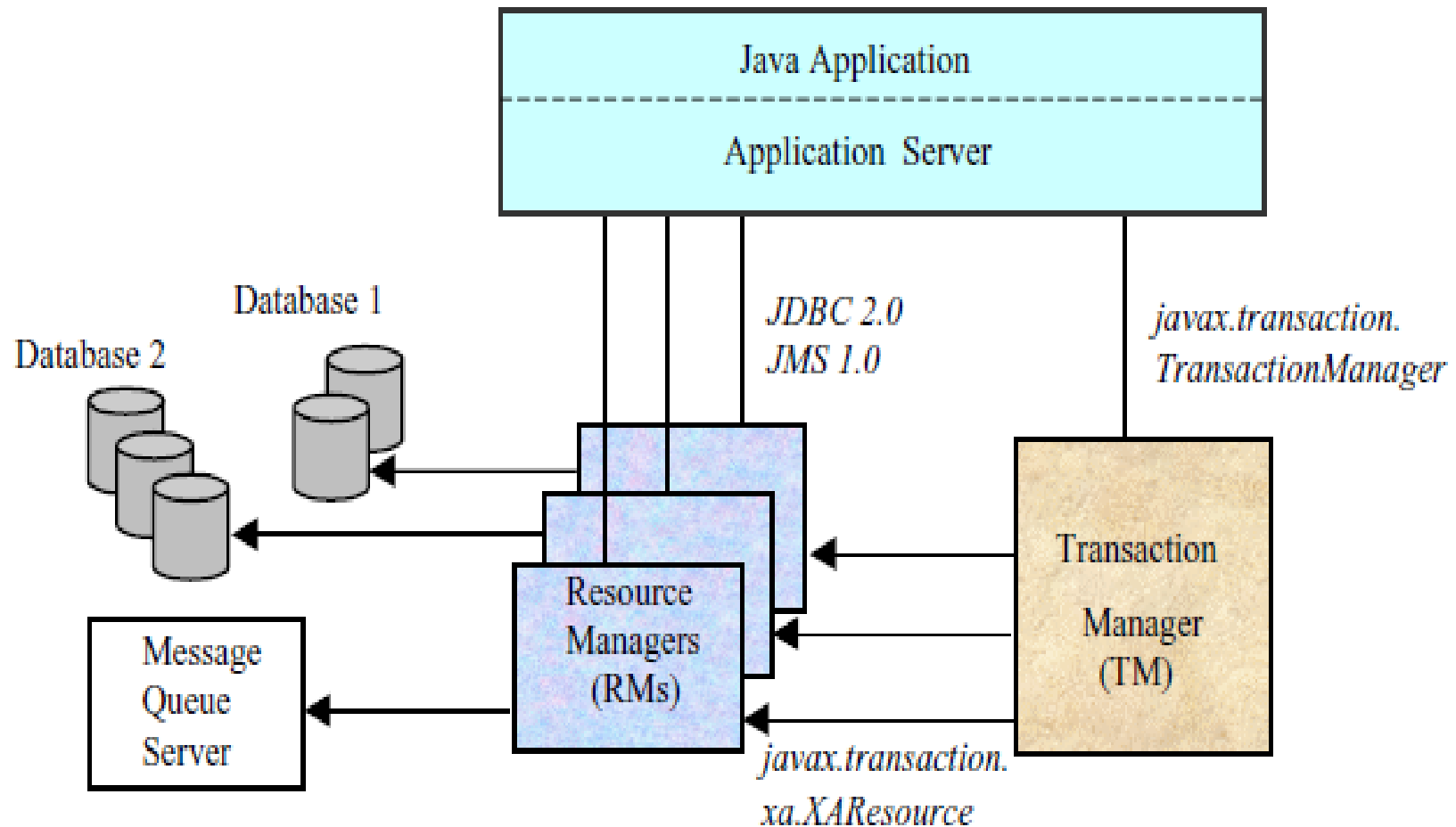
❶ Локальные транзакции

- ❶ Работаем из EJB через JDBC/JMS/специфический протокол с ресурсами напрямую выполняя COMMIT/ROLLBACK
- ❶ Принимаем самостоятельно решение о COMMIT/ROLLBACK

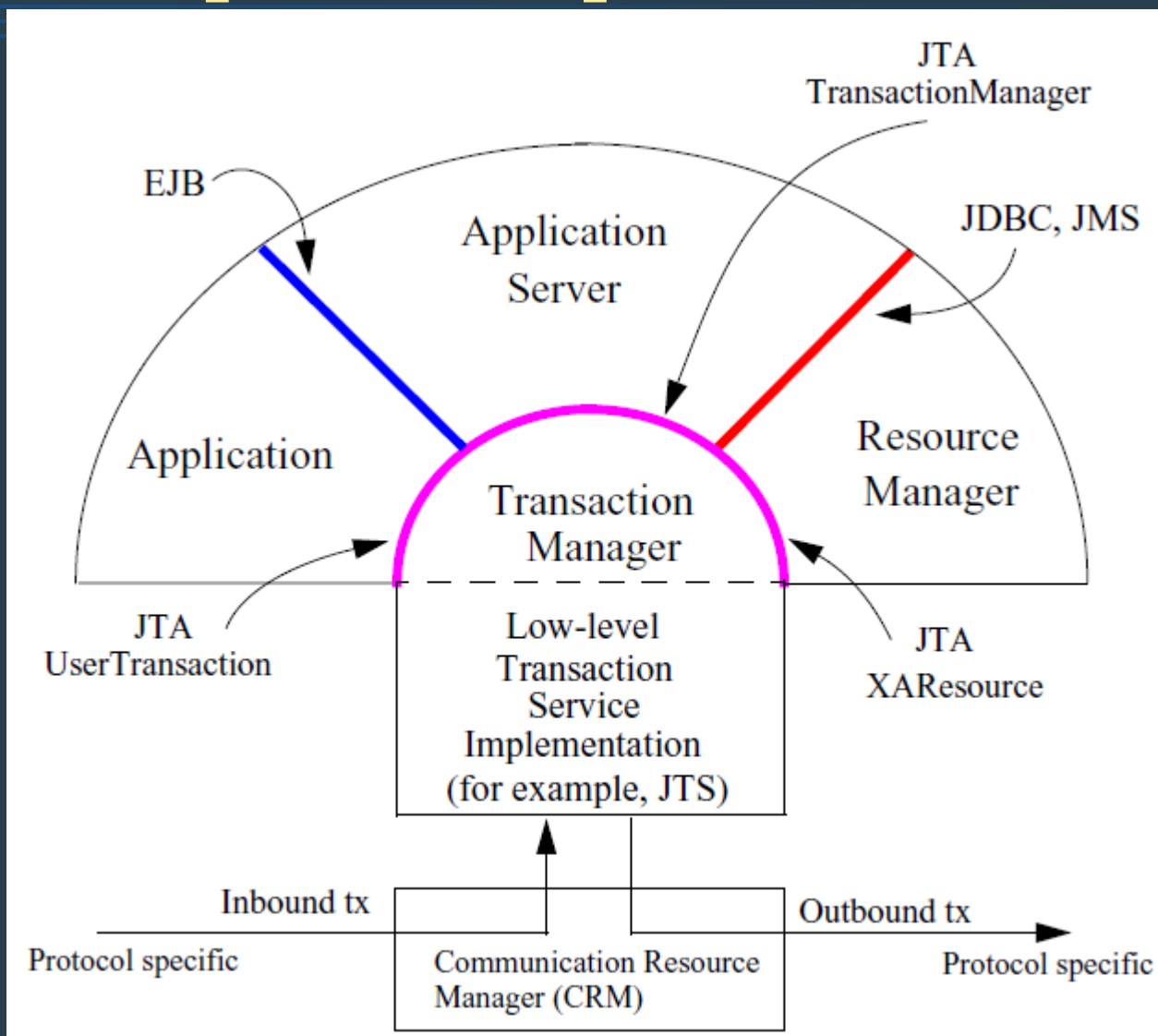
❷ Распределенные транзакции

- ❶ Вовлечено много ресурсов в процессе обработкт бизнес-логики
- ❶ Единый для всех интерфейс для взаимодействия с текущей транзакцией (JTA)
- ❶ UserTransaction
- ❶ Решение о COMMIT/ROLLBACK может приниматься контейнером или классом
- ❶ Все ресурсы должны поддерживать 2-х фазный протокол (кроме одного)(XA-протокол)

Распределенные транзакции



Взаимодействие компонентов при обработке транзакции



Интерфейс TransactionManager

- Интерфейс между сервером приложений и менеджером транзакций

```
interface javax.transaction.TransactionManager
{
    public void begin();
    public void commit();
    public int getStatus();
    public Transaction getTransaction();
    public void resume(Transaction tobj);
    public void rollback();
    public void setRollbackOnly();
    public void setTransactionTimeout(int seconds);
    public Transaction suspend() ;
}
```

Интерфейс Transaction

- ❏ Интерфейс управления транзакцией и ее RM для сервера приложений

```
interface javax.transaction.Transaction
{
    public void commit();
    public boolean delistResource(XAResource xaRes, int flag);
    public boolean enlistResource(XAResource xaRes);
    public int getStatus();
    public void registerSynchronization(Synchronization sync);
    public void rollback();
    public void setRollbackOnly();
}
```

Интерфейс UserTransaction

- Интерфейс управления транзакцией для приложения

```
public interface javax.transaction.UserTransaction
{
    public void begin();
    public void commit();
    public int getStatus();
    public void rollback();
    public void setRollbackOnly();
    public void setTransactionTimeout(int seconds);
}
```

Интерфейс Xid

- Интерфейс доступа к идентификатору транзакции

```
public interface javax.transaction.xa.Xid
{
    int getFormatId();
    byte[] getGlobalTransactionId();
    byte[] getBranchQualifier();
}
```

Интерфейс XAResource

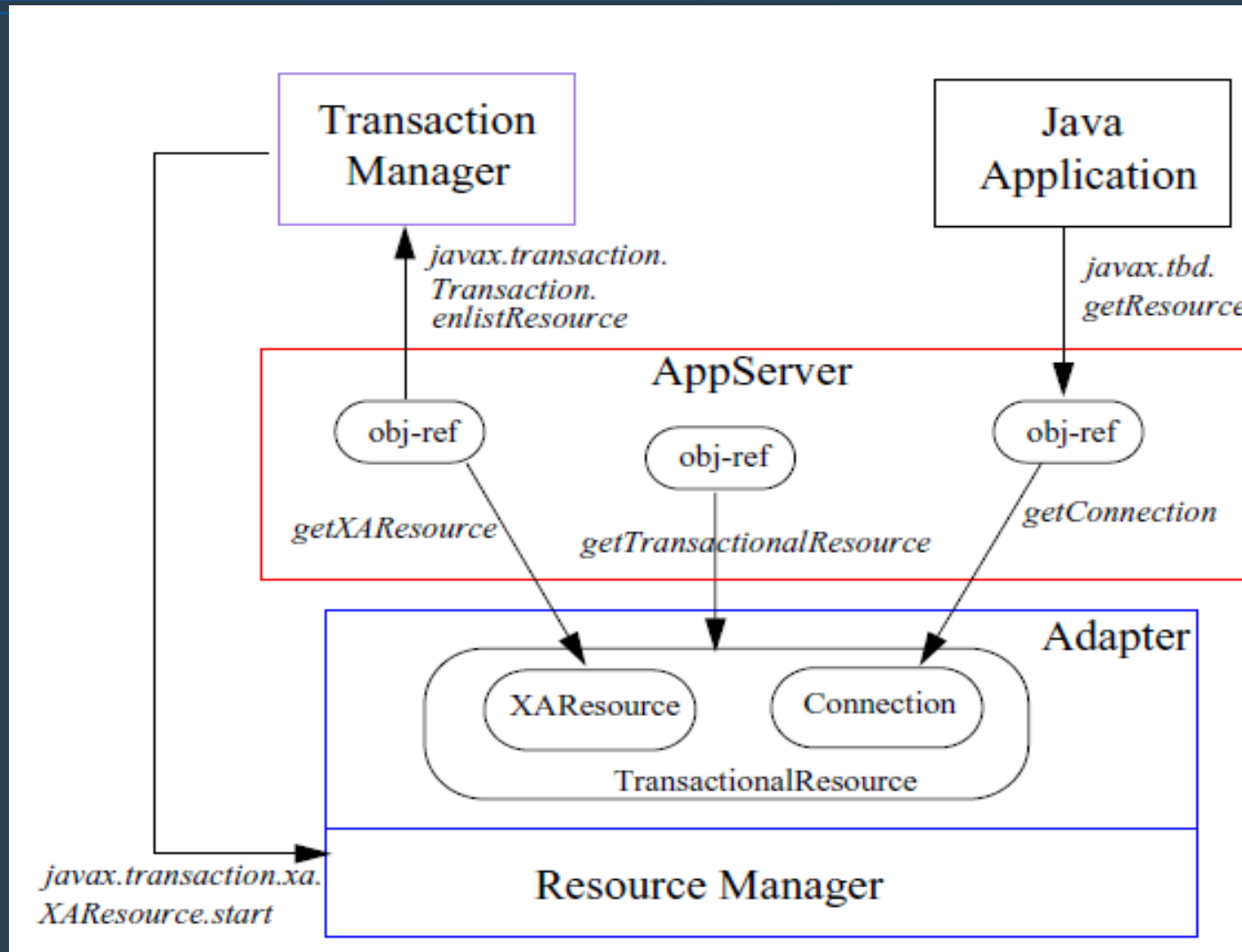
- Транзакционный интерфейс доступа к ресурсам (RM)

```
public interface javax.transaction.xa.XAResource
{
    public void commit(Xid xid, boolean onePhase);
    public void end(Xid xid, int flags);
    public void forget(Xid xid);
    public int getTransactionTimeout();
    public boolean isSameRM(XAResource xares);
    public int prepare(Xid xid);
    public Xid[] recover(int flag);
    public void rollback(Xid xid);
    public boolean setTransactionTimeout(int seconds);
    public void start(Xid xid, int flags);
}
```

Немного о JNDI

- ❑ Java Naming Directory Interface
- ❑ Содержит именованные ссылки на объекты и фабрики объектов
 - ❑ `java:comp/TransactionManager`
 - ❑ `java:comp/UserTransaction`

Получение ресурсов через сервер приложений

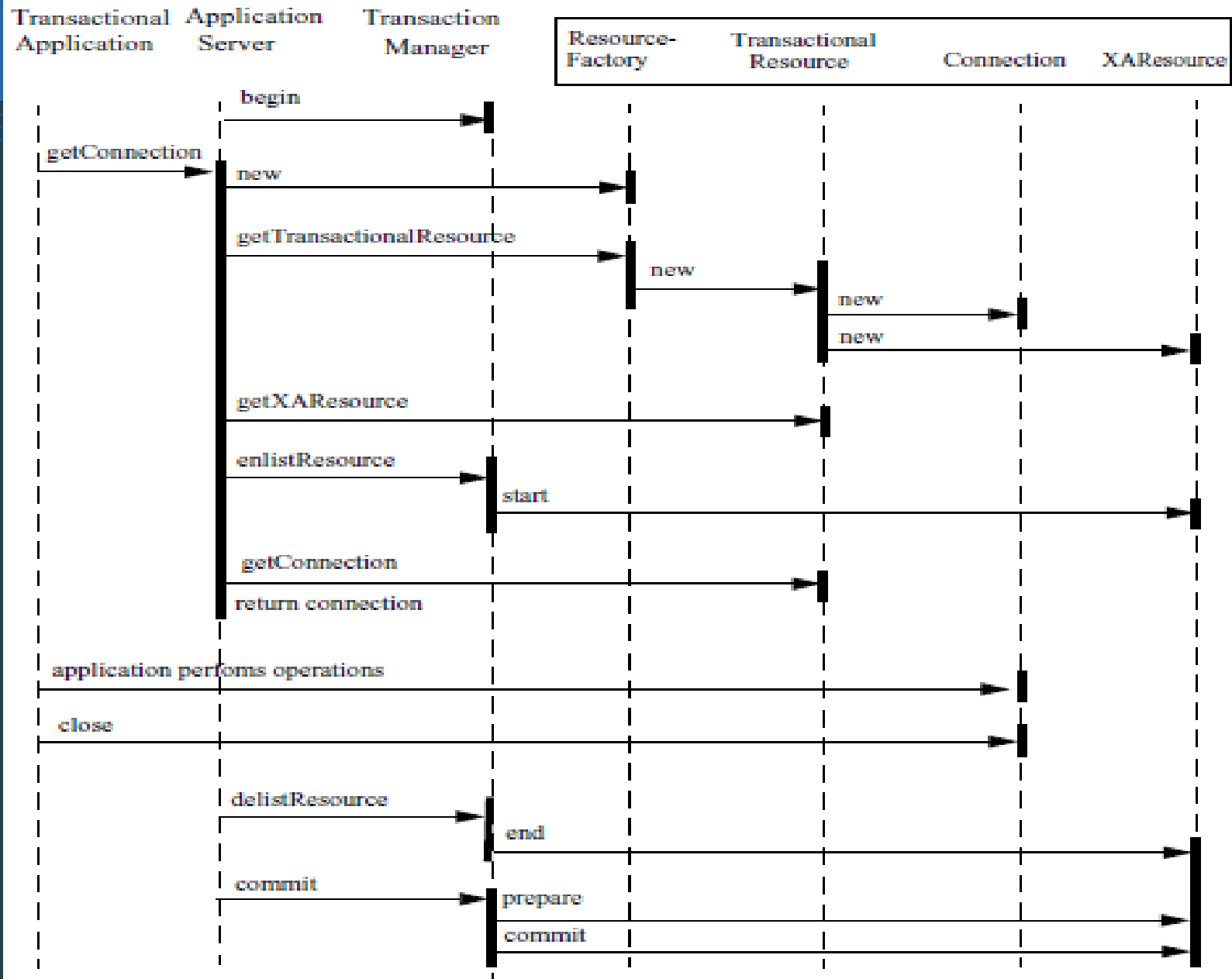


Получение ресурса сервером приложений

```
// Запрос от приложения на получение ресурса/соединения
Context ctx = InitialContext();
ResourceFactory rf =(ResourceFactory)ctx.lookup("MyEISResource");
TransactionalResource res = rf.getTransactionalResource();
// Получение XAResource части ресурса для менеджера транзакций
XAResource xaRes = res.getXAResource();
(TransactionManager.getTransaction()).enlistResource(xaRes);

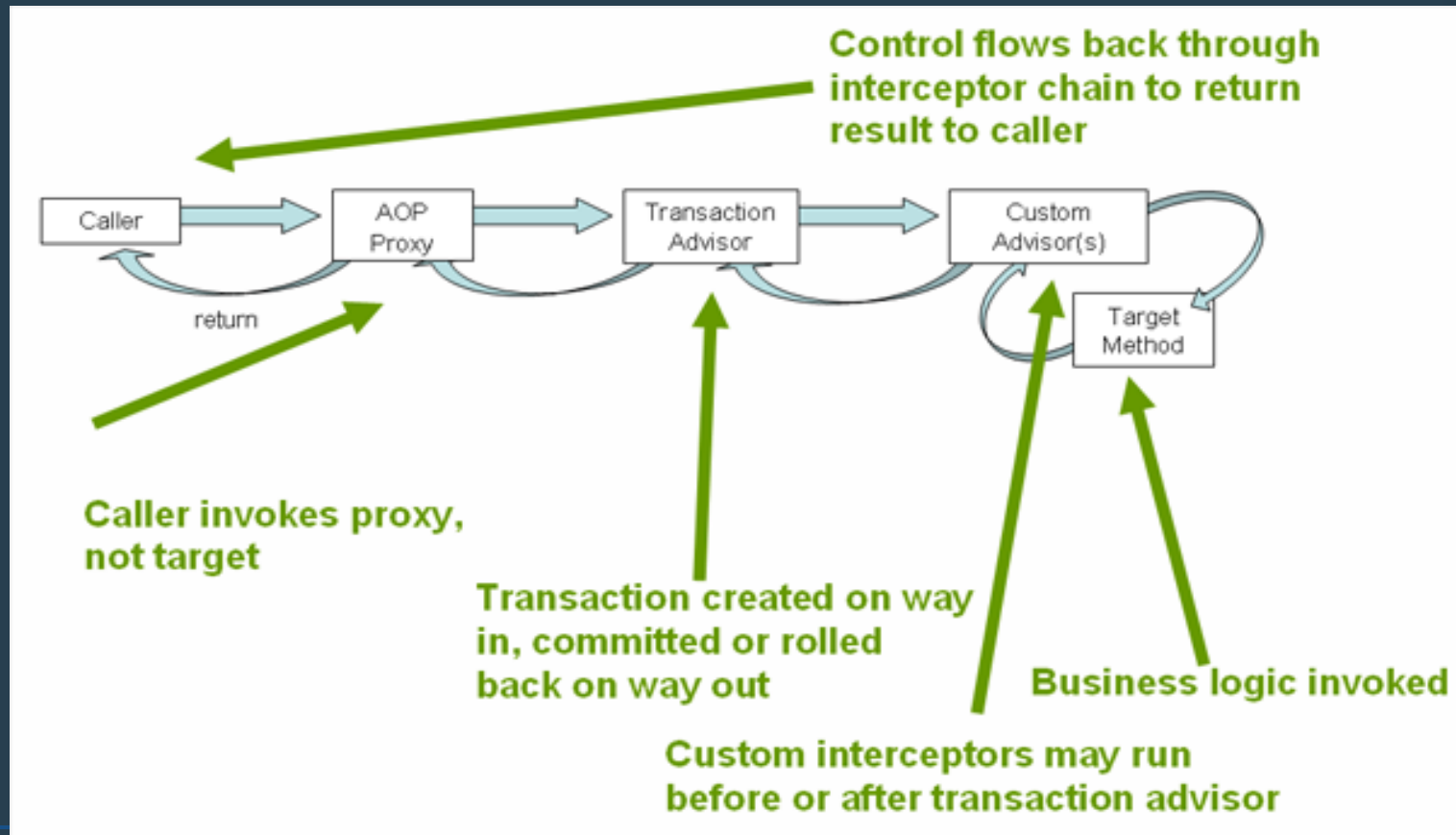
// Получение соединения как части ресурса
Connection con = (Connection)res.getConnection();
return con;
```

Interfaces implemented by the resource adapter



Управление транзакциями в SpringFramework

- Aspects Oriented Programming (AOP)
- Injection-of-Control (IoC)



Пример сервиса

```
interface BookService {  
    void buyBooks(...);  
}
```

```
class DefaultBookService  
{  
    void buyBooks(...);  
}
```

```
class MyPage {  
    BookService bookService;  
  
    {  
        ....  
        ■ bookService.buyBooks(...)  
    }  
}
```

Описание транзакционного сервиса

```
<beans ....>

  <bean id="bookService"
class="x.y.service.DefaultBookService"/>

  <tx:advice id="txAdvice" transaction-
manager="txManager">

    <tx:attributes>

      <tx:method name="get*" read-only="true"/>

      <tx:method name="*"
propagation="REQUIRED"/>

    </tx:attributes>

  </tx:advice>

  <aop:config>

    <aop:pointcut id="bookServiceOperation"
expression="execution(*
x.y.service.BookService.*(..))"/>

    <aop:advisor advice-ref="txAdvice" pointcut-
ref="bookServiceOperation"/>

  </aop:config>
```

```
<bean id="dataSource"
class="org.apache.commons.dbcp.BasicDataSource"
destroy-method="close">

  <property name="driverClassName"
value="oracle.jdbc.driver.OracleDriver"/>

  <property name="url" value="jdbc:oracle:thin:@rj-
t42:1521:elvis"/>

  <property name="username" value="scott"/>

  <property name="password" value="tiger"/>

</bean>

<bean id="txManager"
class="org.springframework.jdbc.datasource.DataSourc
eTransactionManager">

  <property name="dataSource" ref="dataSource"/>

</bean>

</beans>
```



www.pictofigo.com

Вопросы?